

T10 — Variabili, Scope, Call Stack e Memoria Heap

Stack Frame, Passaggio dei Parametri per Valore, Garbage Collection e Debutto di MavenDate

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco analizziamo in profondità il modello di memoria della JVM (Stack vs Heap), il passaggio dei parametri per valore, gli oggetti immutabili e i tipi wrapper ([T10 - Java Variables and Call Stack.pdf](#)). Sul fronte pratico analizziamo il **grande spartiacque architetturale del corso**: il passaggio dal progetto manuale a pacchetti [SimpleDate](#) al progetto ufficiale [MavenDate](#) con Apache Maven, file `pom.xml`, layout `src/main/java` e collisione di nomi tra package diversi.

0.0.1 1. Teoria: Concetti Teorici dalle Slide (Slide T10)

A. Il Modello di Memoria della JVM: Stack vs Heap (Slide 1–11) La memoria a disposizione di un processo Java è partizionata in tre aree principali: 1. **Call Stack (Stack dei Thread)**: - Memoria estremamente veloce, gestita a politica LIFO (Push/Pop). - È suddivisa in **Stack Frame** (o record di attivazione): ogni invocazione di metodo crea un nuovo frame nello stack contenente: - La tabella delle variabili locali. - I parametri formali passati al metodo. - Lo stack degli operandi (per i calcoli intermedi della CPU/bytecode). - I riferimenti alla costante del metodo e l'indirizzo di ritorno. - Quando il metodo termina (con `return` o eccezione), il frame viene rimosso istantaneamente dallo Stack e tutte le sue variabili locali vengono distrutte. 2. **Heap (Memoria Dinamica degli Oggetti)**: - Memoria globale condivisa da tutti i thread dell'applicazione. - Ospita tutti gli **oggetti** istanziati con `new`, inclusi gli **array**. - I dati nello Heap non si deallocano all'uscita dal metodo: sopravvivono finché esiste almeno un riferimento attivo nello Stack o in altri oggetti che punta ad essi. Quando diventano irraggiungibili (*unreachable*), vengono eliminati dal Garbage Collector. 3. **Method Area / Metaspace (Memoria di Classe)**: - Ospita il bytecode compilato delle classi caricate dal Class-Loader, i metadati di tipo e le **variabili statiche** (`static`). - **Blocco di Inizializzazione Statica** (`static { ... }`): - Blocco speciale eseguito **una sola volta** nel ciclo di vita dell'applicazione, al momento in cui la classe viene caricata in memoria dalla JVM: `java static int[] arr; static { arr = new int[3]; arr[0] = 1; arr[1] = 2; arr[2] = 0; }` - Si usa per inizializzare strutture statiche complesse che richiedono cicli o controlli prima dell'uso.

B. Immutabilità e Passaggio Parametri: Pass-by-Value (Slide 15–25)

- **Oggetti Immutabili (*Immutable Objects*)**:
 - Un oggetto il cui stato interno non può essere modificato dopo la costruzione (es. `String`, `Integer`, `Date` da Lezione 09).

- *Regole per creare una classe immutabile:*
 1. Rendere tutti i campi `private final`.
 2. Non fornire metodi mutatori (nessun setter).
 3. Dichiarare la classe `final` (per impedire che le sottoclassi violino l'immutabilità con override malevoli).
 4. Se l'oggetto contiene riferimenti a strutture mutabili (es. un array o un'altra data mutabile), eseguire copie difensive (*defensive copy*) nel costruttore e nei getter.
- *Vantaggi:* Immuni da side-effects da aliasing, intrinsecamente *thread-safe* (possono essere condivisi tra thread concorrenti senza sincronizzazione o lock).

- **Come Java passa i parametri ai metodi: SEMPRE Strictly Pass-by-Value:**

- In Java **non esiste il passaggio per riferimento del C++ (`int& x`)**.
- **Per i tipi primitivi (`int`, `double`):** il metodo riceve una copia del valore binario. Qualsiasi modifica locale sul parametro non ha alcun effetto sulla variabile del chiamante.
- **Per i tipi reference (oggetti/array):** il metodo riceve **una copia per valore del riferimento** (dell'indirizzo puntato):
 - * *Caso 1 (Mutazione dell'oggetto):* se usiamo il riferimento ricevuto per chiamare un metodo mutatore (`p.setAge(25)`), l'oggetto puntato nello Heap viene effettivamente modificato per il chiamante!
 - * *Caso 2 (Riassegnamento del puntatore — Il tranello d'esame):*

```

1 void reset(Person p) {
2     p = new Person("Bob"); // Assegna un nuovo oggetto al puntatore locale!
3 }

```

All'esterno, la variabile del chiamante **non cambia affatto!** Ha semplicemente mutato la copia locale del puntatore nel proprio stack frame.

C. Classi Wrapper, Autoboxing e i loro Trabocchetti (Slide 26–35)

- Per ciascuno degli 8 tipi primitivi, Java mette a disposizione una corrispondente **Wrapper Class** nel package `java.lang`:
 - `byte` → `Byte`
 - `short` → `Short`
 - `int` → `Integer`
 - `long` → `Long`
 - `float` → `Float`
 - `double` → `Double`
 - `char` → `Character`
 - `boolean` → `Boolean`
- Tutte le classi wrapper sono **immutabili**.
- **Autoboxing e Unboxing (Java 5+):**
 - *Autoboxing:* conversione automatica da primitivo a wrapper (`Integer x = 5;` ⇒ compilato come `Integer.valueOf(5)`).
 - *Unboxing:* estrazione automatica del valore primitivo dal wrapper (`int y = x;` ⇒ compilato come `x.intValue()`).
- Attenzione: **I Due Grandi Pericoli delle Wrapper Classes:**
 1. **Il Trabocchetto dell'Integer Cache ($-128 \dots + 127$):**

```

1 Integer a = 100, b = 100;
2 System.out.println(a == b); // TRUE! (La JVM ricicla lo stesso oggetto cacheato)
3
4 Integer c = 200, d = 200;
5 System.out.println(c == d); // FALSE! (Fuori range [-128, 127], alloca 2 oggetti distinti
  nello Heap!)

```

Conclusione: **Mai usare == per confrontare i wrapper**, usare sempre `.equals()`!

2. Crash da Unboxing su null:

```

1 Integer obj = null;
2 int val = obj; // RUNTIME ERROR: NullPointerException!

```

La JVM tenta di invocare `obj.intValue()`, fallendo rovinosamente se `obj` è nullo.

0.0.2 2. Codice: Evoluzione del Codice: Il Passaggio a MavenDate

In [Lezione10](#), il docente archivia il vecchio progetto non strutturato e adotta lo standard industriale **Apache Maven**.

1. Riorganizzazione dei Package Il progetto viene suddiviso in due package distinti: - **it.oop.core**: il dominio applicativo (le classi di modello: `Date.java`, `Language.java`, `BirthDay.java`). - **it.oop.ui**: l'interfaccia utente contenente il main (`MainDate.java`).

2. Il Test Didattico sulla Collisione dei Nomi di Package Nel package `it.oop.ui`, il docente aggiunge appositamente una seconda classe: `it/oop/ui/Date.java`:

```

1 package it.oop.ui;
2
3 class Date { // Visibilità package-private
4     public int time;
5     private int start = 0;
6
7     public Date(int time) { this.time = time; }
8 }

```

E in `MainDate.java` mostra come la JVM risolve l'ambiguità:

```

1 package it.oop.ui;
2
3 import it.oop.core.Date; // 1. L'import esplicito ha la precedenza!
4
5 public class MainDate {
6     public static void main(String[] args) {
7         Date d1 = new Date(7, 1, 2025); // Istanza it.oop.core.Date!
8
9         // 2. Per istanziare la classe Date del package it.oop.ui,
10        // è obbligatorio usare il Fully Qualified Name (FQN):
11        it.oop.ui.Date d = new it.oop.ui.Date(12345);
12        System.out.println(d.time); // OK: 'time' è public
13        // System.out.println(d.start); // COMPILE-TIME ERROR: 'start' è private!
14    }
15 }

```

3. Anatomia del `pom.xml` di MavenDate

```

1 <project ...>
2   <modelVersion>4.0.0</modelVersion>
3   <groupId>it.oop</groupId>
4   <artifactId>MavenDate</artifactId>
5   <version>1.0-SNAPSHOT</version>
6
7   <properties>
8     <maven.compiler.source>17</maven.compiler.source>
9     <maven.compiler.target>17</maven.compiler.target>
10    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
11  </properties>
12
13  <build>
14    <plugins>
15      <!-- Plugin per creare il JAR eseguibile con dipendenze -->
16      <plugin>
17        <groupId>org.apache.maven.plugins</groupId>
18        <artifactId>maven-assembly-plugin</artifactId>
19        <version>3.7.1</version>
20        <configuration>
21          <descriptorRefs>
22            <descriptorRef>jar-with-dependencies</descriptorRef>
23          </descriptorRefs>
24          <archive>
25            <manifest>
26              <addClasspath>>true</addClasspath>
27              <!-- Entrypoint del JAR -->
28              <mainClass>it.oop.ui.MainDate</mainClass>
29            </manifest>
30          </archive>
31        </configuration>
32        <executions>
33          <execution>
34            <id>assemble-all</id>
35            <phase>package</phase>
36            <goals>
37              <goal>single</goal>
38            </goals>
39          </execution>
40        </executions>
41      </plugin>
42    </plugins>
43  </build>
44 </project>

```

Questo file è il prototipo esatto che hai esteso in tutti i moduli del tuo workspace `esercizi-wally-25-26`:

- `compile`: compila da `src/main/java` verso `target/classes`.
- `test`: compila ed esegue i test da `src/test/java` verso `target/test-classes`.
- `package`: genera il JAR auto-eseguibile con `META-INF/MANIFEST.MF` configurato con `Main-Class: it.oop.ui.MainDate`.

0.0.3 3. Ciclo: Corrispondenza con i Tuoi Moduli Workspace

Nel tuo repository `esercizi-wally-25-26`:

- **es08**: `Memory.java` implementa l’allocazione Stack vs Heap e il blocco static `{ ... }`.
- **es10**: `ImmutablePerson.java` applica formalmente tutte le regole di immutabilità di T10 (final class, campi final, validazione e assenza di setter).

Nota di Studio

Con la **Lezione 10** abbiamo completato l'intero quadro su memoria, call stack, tipi wrapper e la transizione a Maven.

Il prossimo blocco è la **Lezione 11 / Slide T11: *Packages and Class Visibility***: - Tassonomia gerarchica dei package e convenzioni DNS invertite (`it.univr...`). - Regole di visibilità e accessibilità inter-package: membri `public`, `protected`, `package-private` e `private`. - Evoluzione di `MavenDate`: - Refactoring dell'algoritmo Gregoriano: introduzione definitiva del **calcolo dell'anno bisestile (`isLeapYear`)** in `Date.java`! - Integrazione di `isLeapYear(year)` in `daysPerMonth(month, year)` e fine dei mesi errati per Febbraio!

Dammi conferma per procedere alla Lezione 11 / T11!